

第六章 多线程并发（面试突击★★★★★）

多线程和并发是C++后台开发面试的绝对重点，尤其是原子操作、内存序、线程池、死锁预防等，属于高频考点。

目录

- 1. [线程基础](#)
- 2. [线程同步](#)
- 3. [死锁](#)
- 4. [锁优化](#)
- 5. [原子操作](#)
- 6. [Future与Promise](#)
- 7. [线程池](#)
- 8. [生产者消费者模型](#)
- 9. [并发数据结构](#)
- 10. [面试总结](#)

1. 线程基础

1.1 线程 vs 进程

特性	进程	线程
定义	资源分配的基本单位	CPU调度的基本单位
地址空间	独立虚拟地址空间	共享进程地址空间
上下文切换	慢(TLB/页表/缓存)	快(只需切换寄存器/栈)
创建销毁开销	大(fork约数百微秒)	小(pthread约10微秒)
通信方式	管道/共享内存/消息队列	共享内存/条件变量/原子
内存隔离	进程间隔离	同一进程内无隔离
崩溃影响	不影响其他进程	可能导致进程崩溃
数据共享	需要IPC, 开销大	直接共享, 需同步保护
系统调用	fork/exec/wait	pthread_create等
内核数据结构	task_struct + mm_struct	task_struct(共享mm_struct)

每个进程至少有一个线程（主线程）

线程独享：

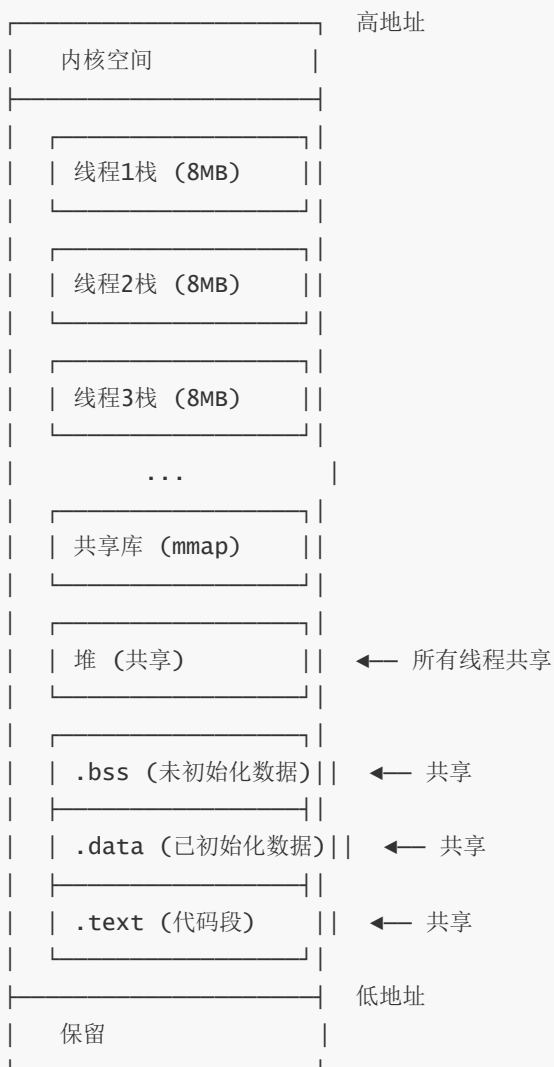
- 线程ID
- 栈（Stack）
- 寄存器（包括PC，SP）
- 信号掩码

- `errno`
- 线程局部存储 (`thread_local`)

线程共享：

- 代码段 (`.text`)
- 数据段 (`.data`, `.bss`)
- 堆 (`Heap`)
- 打开的文件描述符
- 信号处理函数
- 当前工作目录
- 用户ID/组ID

线程的内存布局：



1.2 C++11 `std::thread` 完整示例

```
#include <iostream>
#include <thread>
#include <string>
#include <chrono>
#include <vector>
#include <functional>
```

```
// ===== 基础示例 =====

// 1. 普通函数作为线程入口
void thread_func(int id, const std::string& msg) {
    std::cout << "Thread " << id << ": " << msg
                << " (tid=" << std::this_thread::get_id() << ")"
                << std::endl;
}

// 2. 函数对象 (Functor) 作为线程入口
struct Task {
    void operator()(int n) const {
        for (int i = 0; i < n; i++) {
            std::cout << "Task iteration " << i << std::endl;
            std::this_thread::sleep_for(std::chrono::milliseconds(100));
        }
    }
};

// 3. Lambda 作为线程入口 (最常用)
void lambda_example() {
    int shared_data = 42;

    std::thread t([&shared_data]() {
        std::cout << "Lambda thread: shared_data = "
                  << shared_data << std::endl;
        shared_data = 100;
    });

    t.join();
    std::cout << "After thread: shared_data = " << shared_data << std::endl;
}

// 4. 成员函数作为线程入口
class Worker {
public:
    void process(int id) {
        std::cout << "Worker::process id=" << id << std::endl;
    }

    static void static_process(int id) {
        std::cout << "Worker::static_process id=" << id << std::endl;
    }
};

void member_function_example() {
    Worker w;

    // 非静态成员函数需要传入对象指针 (或ref)
    std::thread t1(&Worker::process, &w, 1);

    // 静态成员函数

```

```

    std::thread t2(&worker::static_process, 2);

    t1.join();
    t2.join();
}

int main() {
    // 创建线程的几种方式
    std::thread t1(thread_func, 1, "Hello from function");
    std::thread t2(Task(), 3); // 函数对象
    std::thread t3([]() { // Lambda
        std::cout << "Lambda thread running" << std::endl;
    });

    t1.join();
    t2.join();
    t3.join();

    lambda_example();
    member_function_example();

    return 0;
}

```

1.3 线程生命周期：创建、join、detach

```

#include <iostream>
#include <thread>
#include <chrono>

// ===== join() =====
// 等待线程完成执行，阻塞调用线程
void join_example() {
    std::thread t([]() {
        std::this_thread::sleep_for(std::chrono::seconds(2));
        std::cout << "Worker done" << std::endl;
    });

    std::cout << "Waiting for thread..." << std::endl;
    t.join(); // 阻塞直到t完成
    std::cout << "Thread joined" << std::endl;
    // 此时t.joinable() == false
}

// ===== detach() =====
// 分离线程，线程在后台独立运行
// 线程完成时资源由C++运行时库回收
void detach_example() {
    std::thread t([]() {
        std::this_thread::sleep_for(std::chrono::seconds(1));
        std::cout << "Detached thread done" << std::endl;
    });
}

```

```

    t.detach(); // 分离线程
    // 此时t.joinable() == false
    // 不能对detach的线程调用join

    std::cout << "Main continuing..." << std::endl;
    // 注意：如果main退出太快，detach的线程可能还没执行完
    // 需要确保进程不会提前退出
    std::this_thread::sleep_for(std::chrono::seconds(2));
}

// ===== joinable() =====
// 检查线程是否可join（即是否关联着一个活跃线程）
void joinable_example() {
    std::thread t;
    std::cout << "Default-constructed: " << t.joinable() << std::endl; // 0

    t = std::thread([]() {});
    std::cout << "After assignment: " << t.joinable() << std::endl; // 1

    t.join();
    std::cout << "After join: " << t.joinable() << std::endl; // 0
}

// ===== RAII线程管理 =====
// 确保线程在异常情况下也能正确join
class ThreadGuard {
    std::thread& t;
public:
    explicit ThreadGuard(std::thread& t_) : t(t_) {}
    ~ThreadGuard() {
        if (t.joinable()) {
            t.join(); // 确保线程被join
        }
    }
    ThreadGuard(const ThreadGuard&) = delete;
    ThreadGuard& operator=(const ThreadGuard&) = delete;
};

void raii_example() {
    std::thread t([]() {
        std::this_thread::sleep_for(std::chrono::seconds(1));
    });

    ThreadGuard guard(t); // RAII: 即使抛异常也会join

    // 可能抛出异常的操作
    // do_something_that_may_throw();
} // guard析构 → t.join()

```

1.4 thread_local 存储

```
#include <iostream>
#include <thread>
#include <mutex>

/*
 * thread_local 变量:
 *   - 每个线程拥有一份独立的副本
 *   - 生命周期: 线程开始时创建, 线程结束时销毁
 *   - 类似全局变量但每个线程独立
 *   - C++11 关键字
 */

// 每个线程都有自己独立的 counter
thread_local int tls_counter = 0;

// 模拟每个线程的日志上下文
thread_local char thread_name[64] = "unknown";

void thread_func(const char* name) {
    // 设置线程本地的名称
    snprintf(thread_name, sizeof(thread_name), "%s", name);

    for (int i = 0; i < 5; i++) {
        tls_counter++;
        printf("[%s] counter = %d\n", thread_name, tls_counter);
    }
}

void thread_local_example() {
    std::thread t1(thread_func, "worker-1");
    std::thread t2(thread_func, "worker-2");

    t1.join();
    t2.join();
}

/*
 * thread_local 常见用途:
 * 1. 线程级日志上下文 (线程ID、请求ID)
 * 2. 线程级随机数生成器
 * 3. 避免锁竞争的高频计数器 (线程本地累加, 定期归并)
 * 4. errno (POSIX中errno通常是线程局部的)
 *
 * 注意:
 * - thread_local 变量的地址不是常量 (不同线程中不同)
 * - 动态初始化的thread_local变量在首次使用时初始化
 * - 线程创建时不复制父线程的thread_local值
 */
```

1.5 线程安全定义

线程安全 (Thread Safety):

一个函数/类是线程安全的，当且仅当：

- 1. 多个线程同时调用时，行为总是正确的
- 2. 不需要调用者做额外的同步

线程安全级别：

不可变 (Immutable)	天生线程安全，如const对象、不变对象
线程安全	通过锁/原子操作保证并发正确性
条件安全	部分方法是线程安全的，部分不是
非线程安全	多线程调用会导致数据竞争
线程对立	即使外部加锁也无法安全使用

常见线程安全问题：

- 1. 数据竞争 (Data Race)：两个线程同时访问同一内存，至少一个是写
- 2. 竞态条件 (Race Condition)：执行结果取决于线程的调度顺序
- 3. ABA问题：CAS操作中，值从A→B→A，CAS误判为未变化
- 4. 死锁 (Deadlock)：多个线程互相等待对方释放资源

保证线程安全的手段：

- 1. 互斥锁 (mutex) - 保证互斥访问
- 2. 原子操作 (atomic) - 无锁的原子操作
- 3. 不可变对象 - 不修改的数据天然安全
- 4. 线程局部存储 - 每个线程独立数据
- 5. 无共享架构 - 如消息传递模型 (Actor model)

2. 线程同步

2.1 std::mutex - 互斥锁

```
#include <iostream>
#include <thread>
#include <mutex>
#include <vector>

/*
 * std::mutex:
 *   - 不可拷贝，不可移动
 *   - lock(): 获取锁，如果锁已被持有则阻塞
 *   - unlock(): 释放锁
 *   - try_lock(): 尝试获取锁，立即返回bool
 *
 * 内部实现：
 *   基于futex (Fast Userspace muTEX) 系统调用
 *   无竞争时：用户态原子操作即可获取锁（快）
 *   有竞争时：通过futex系统调用进入内核等待（慢）
 */
```

```

*/

// ===== 基本用法 =====

int shared_counter = 0;
std::mutex mtx;

void increment(int n) {
    for (int i = 0; i < n; i++) {
        mtx.lock();
        shared_counter++;    // 临界区
        mtx.unlock();
    }
}

void basic_mutex_example() {
    std::vector<std::thread> threads;
    for (int i = 0; i < 10; i++) {
        threads.emplace_back(increment, 100000);
    }
    for (auto& t : threads) t.join();

    std::cout << "Final counter: " << shared_counter << std::endl;
    // 正确结果: 1000000
}

```

2.2 std::recursive_mutex - 递归锁

```

#include <mutex>

/*
 * std::recursive_mutex:
 *   同一个线程可以多次lock同一个递归锁
 *   需要匹配次数的unlock才能最终释放
 *
 * 使用场景: 递归调用、同一个线程中嵌套调用需要加锁的函数
 *
 * 警告: 大多数情况下递归锁是设计不良的信号
 *       如果发现自己需要递归锁, 应该考虑重构代码
 */

class RecursiveExample {
    std::recursive_mutex mtx;
    int value = 0;

public:
    void set(int v) {
        std::lock_guard<std::recursive_mutex> lock(mtx);
        value = v;
    }

    int get() {

```



```

        std::lock_guard<std::recursive_mutex> lock(mtx);
        return value;
    }

    void set_and_get() {
        std::lock_guard<std::recursive_mutex> lock(mtx);
        value = 42;
        // 在已经持有锁的情况下再次调用需要加锁的方法
        int v = get(); // 递归获取同一把锁 - 不会死锁!
        std::cout << "value = " << v << std::endl;
    }
};

```

2.3 std::timed_mutex - 超时锁

```

#include <mutex>
#include <chrono>

/*
 * std::timed_mutex:
 * 增加了超时功能的互斥锁
 * try_lock_for(duration): 等待指定时间段
 * try_lock_until(timepoint): 等待到指定时间点
 */

std::timed_mutex tmtx;

void timed_lock_example() {
    // 尝试获取锁, 最多等100ms
    if (tmtx.try_lock_for(std::chrono::milliseconds(100))) {
        // 获取到锁
        // ...处理...
        tmtx.unlock();
    } else {
        std::cout << "Failed to acquire lock within 100ms" << std::endl;
    }

    // 尝试获取锁, 等到某个时间点
    auto deadline = std::chrono::steady_clock::now()
        + std::chrono::seconds(5);
    if (tmtx.try_lock_until(deadline)) {
        // ...处理...
        tmtx.unlock();
    }
}

```

2.4 std::shared_mutex - 读写锁 (C++17)

```
#include <shared_mutex>
#include <iostream>
#include <thread>
#include <vector>

/*
 * std::shared_mutex (读写锁):
 *
 * 读-读: 可并发 (共享锁 shared_lock)
 * 读-写: 互斥
 * 写-写: 互斥
 *
 * 适用于: 读多写少的场景
 *
 * 方法:
 *   lock() / unlock()          - 独占锁(写锁)
 *   lock_shared() / unlock_shared() - 共享锁(读锁)
 *
 * RAII包装:
 *   std::lock_guard<std::shared_mutex>    - 独占(写)
 *   std::shared_lock<std::shared_mutex>   - 共享(读)
 */

class ThreadSafeCache {
private:
    mutable std::shared_mutex mtx;
    std::map<int, std::string> data;

public:
    // 写操作: 独占锁
    void put(int key, const std::string& value) {
        std::lock_guard<std::shared_mutex> lock(mtx);
        data[key] = value;
    }

    // 读操作: 共享锁 (多个线程可同时读)
    std::string get(int key) const {
        std::shared_lock<std::shared_mutex> lock(mtx);
        auto it = data.find(key);
        if (it != data.end()) {
            return it->second;
        }
        return "";
    }
};

void shared_mutex_example() {
    ThreadSafeCache cache;

    cache.put(1, "Hello");
}
```

```

cache.put(2, "world");

// 多个读线程可以并发执行
std::vector<std::thread> readers;
for (int i = 0; i < 10; i++) {
    readers.emplace_back([&cache]() {
        for (int j = 0; j < 1000; j++) {
            cache.get(1); // 并发读，不阻塞
        }
    });
}
for (auto& t : readers) t.join();
}

```

2.5 std::lock_guard - RAII锁

```

/*
 * std::lock_guard:
 *   - 构造时 lock()
 *   - 析构时 unlock()
 *   - 不可拷贝，不可移动
 *   - 最简单、最轻量的RAII锁
 *
 * 优点：
 *   1. 异常安全：即使抛异常也会自动解锁
 *   2. 避免忘记unlock
 *   3. 0开销：编译后与手动lock/unlock一样的汇编
 */

void lock_guard_example() {
    std::mutex mtx;

    // === 错误做法（手动lock/unlock）===
    // mtx.lock();
    // // ...如果这里抛异常，锁永远不会释放！
    // mtx.unlock();

    // === 正确做法（lock_guard）===
    {
        std::lock_guard<std::mutex> lock(mtx);
        // 临界区代码
        // 即使抛异常，lock析构时也会自动unlock
    } // lock析构 → mtx.unlock()
}

```

2.6 std::unique_lock - 灵活锁

```

#include <mutex>

/*
 * std::unique_lock vs std::lock_guard:

```

```

*
* lock_guard:
*   - 构造时一定lock，析构时一定unlock
*   - 不可拷贝，不可移动
*   - 最轻量，无额外开销
*
* unique_lock:
*   - 可延迟锁定 (defer_lock)
*   - 可尝试锁定 (try_to_lock)
*   - 可接管已锁定的锁 (adopt_lock)
*   - 可手动lock/unlock
*   - 可移动 (move)
*   - 支持条件变量 (condition_variable 必需 unique_lock)
*   - 有轻微额外开销 (需要存储锁状态)
*/

std::mutex global_mtx;

// 1. 延迟锁定
void defer_lock_example() {
    std::unique_lock<std::mutex> lock(global_mtx, std::defer_lock);
    // 此时锁未被获取
    // ...做一些不需要锁的事情...
    lock.lock();    // 现在才获取锁
    // ...临界区...
    lock.unlock();  // 手动释放
    lock.lock();    // 可以重新获取
    // ...更多临界区...
} // 如果lock仍然持有锁，析构时自动unlock

// 2. 尝试锁定
void try_lock_example() {
    std::unique_lock<std::mutex> lock(global_mtx, std::try_to_lock);
    if (lock.owns_lock()) {
        // 获取到了锁
        std::cout << "Got the lock" << std::endl;
    } else {
        // 锁被其他人持有
        std::cout << "Lock is held by another thread" << std::endl;
    }
}

// 3. 转移所有权
std::unique_lock<std::mutex> create_lock() {
    std::unique_lock<std::mutex> lock(global_mtx);
    // ...做一些初始化工作...
    return lock; // 移动语义，转移所有权给调用者
}

void transfer_ownership_example() {
    std::unique_lock<std::mutex> lock = create_lock();
    // 此时调用者持有锁
    // ...继续工作...
}

```

```
}
```

2.7 std::scoped_lock - 多锁同时获取 (C++17)

```
#include <mutex>

/*
 * std::scoped_lock (C++17):
 * 同时获取多个互斥锁，避免死锁
 * 内部使用 std::lock() 算法（死锁避免）
 *
 * 等价于：
 *     std::lock(m1, m2, ..., mN); // 原子地同时获取
 *     std::lock_guard 组合
 */

class BankAccount {
    std::mutex mtx;
    double balance;
public:
    BankAccount(double initial) : balance(initial) {}

    // 错误做法：可能死锁
    void transfer_wrong(BankAccount& to, double amount) {
        std::lock_guard<std::mutex> lock_from(mtx); // 先锁自己
        std::lock_guard<std::mutex> lock_to(to.mtx); // 再锁对方
        // 如果两个线程同时互相转账：
        // 线程A: lock(a.mtx) → 等待 b.mtx
        // 线程B: lock(b.mtx) → 等待 a.mtx
        // 死锁！
        balance -= amount;
        to.balance += amount;
    }

    // 正确做法：scoped_lock
    void transfer(BankAccount& to, double amount) {
        // 同时获取两把锁，内部使用避免死锁的算法
        std::scoped_lock lock(mtx, to.mtx);
        balance -= amount;
        to.balance += amount;
    }
};
```

2.8 std::condition_variable - 条件变量 ★★★

```
#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variable>
#include <queue>
```

```

/*
 * 条件变量：用于线程间的等待/通知机制
 *
 * 核心方法：
 *   wait(lock):           等待通知，释放锁并睡眠
 *   wait(lock, predicate): 等价于 while(!pred()) wait(lock);
 *   notify_one():         唤醒一个等待线程
 *   notify_all():         唤醒所有等待线程
 *
 * wait的等价代码：
 *   template<typename Predicate>
 *   void wait(std::unique_lock<std::mutex>& lock, Predicate pred) {
 *       while (!pred()) {
 *           wait(lock);
 *       }
 *   }
 */

// ===== 生产者-消费者示例 =====

std::mutex mtx_cv;
std::condition_variable cv;
std::queue<int> data_queue;
bool done = false;

// 生产者
void producer(int id, int num_items) {
    for (int i = 0; i < num_items; i++) {
        std::this_thread::sleep_for(std::chrono::milliseconds(10));
        {
            std::lock_guard<std::mutex> lock(mtx_cv);
            data_queue.push(i);
            std::cout << "Producer " << id << " produced: " << i << std::endl;
        }
        cv.notify_one(); // 通知消费者（在释放锁后通知更优）
    }
    {
        std::lock_guard<std::mutex> lock(mtx_cv);
        done = true;
    }
    cv.notify_all();
}

// 消费者 - 使用wait带谓词（推荐方式）
void consumer(int id) {
    while (true) {
        std::unique_lock<std::mutex> lock(mtx_cv);

        // 带谓词的wait：等价于自动处理虚假唤醒
        cv.wait(lock, [] { return !data_queue.empty() || done; });

        if (!data_queue.empty()) {
            int value = data_queue.front();

```

```

        data_queue.pop();
        lock.unlock(); // 处理数据前释放锁
        std::cout << "Consumer " << id << " consumed: " << value << std::endl;
    } else if (done) {
        break;
    }
}

}

}

void condition_variable_example() {
    std::thread prod(producer, 1, 10);
    std::thread cons1(consumer, 1);
    std::thread cons2(consumer, 2);

    prod.join();
    cons1.join();
    cons2.join();
}

```

虚假唤醒 (Spurious Wakeup)

```

/*
 * 虚假唤醒 (Spurious wakeup):
 *   条件变量的wait可能在没有notify的情况下返回
 *   这是POSIX允许的行为（由于实现和性能优化原因）
 *
 * 为什么有虚假唤醒？
 *   1. 硬件中断/信号可能错误地唤醒等待队列
 *   2. 操作系统调度器的实现细节
 *   3. 简化条件变量的实现
 *
 * 正确处理：总是在循环中检查条件
 */

// 错误做法：if而不是while
void wrong_wait(std::unique_lock<std::mutex>& lock) {
    if (queue.empty()) {           // 只用if检查一次
        cv.wait(lock);             // 虚假唤醒后不会重新检查条件
    }

    // 如果虚假唤醒，队列可能为空，导致错误
    auto item = queue.front();     // 可能访问空队列！
}

// 正确做法：while循环
void correct_wait(std::unique_lock<std::mutex>& lock) {
    while (queue.empty()) {        // while循环
        cv.wait(lock);             // 醒来后重新检查条件
    }

    // 此时队列一定非空
    auto item = queue.front();
}

```

```
// 最佳做法：使用带谓词的wait（自动while循环）
void best_wait(std::unique_lock<std::mutex>& lock) {
    cv.wait(lock, [] { return !queue.empty(); }); // C++帮你写好了while
}
```

丢失唤醒 (Lost Wakeup)

```
/*
 * 丢失唤醒 (Lost Wakeup):
 * 通知发送时没有等待者，通知"丢失"
 *
 * 场景：
 * 线程A：检查条件(不满足) → (上下文切换) → wait
 * 线程B：修改条件(满足) → notify → (但A还没进入wait!)
 * 线程A：进入wait → 永远等待
 *
 * 解决方法：
 * 条件变量的wait必须在持有锁的情况下调用
 * wait内部原子地释放锁并进入等待状态
 * (这就是为什么wait需要传入lock)
 *
 * wait内部大致实现：
 * void wait(std::unique_lock<std::mutex>& lock) {
 *     // 1. 将当前线程加入等待队列
 *     add_to_wait_queue(this_thread);
 *     // 2. 原子地：释放锁 + 设置线程状态为睡眠
 *     atomic { lock.unlock(); set_thread_state(SLEEPING); }
 *     // 3. 被唤醒后重新获取锁
 *     lock.lock();
 * }
 * // 步骤2的原子性保证了不会丢失唤醒
 */
```

2.9 std::counting_semaphore / std::binary_semaphore (C++20)

```
#include <semaphore>

/*
 * C++20 信号量
 *
 * std::counting_semaphore<N>:
 * - 内部计数器，初始值0到N之间
 * - acquire(): 递减计数器，如果为0则阻塞
 * - release(): 递增计数器，唤醒等待者
 *
 * std::binary_semaphore:
 * - counting_semaphore<1> 的别名
 * - 只有0和1两个状态
 * - 类似互斥锁但release和acquire可以在不同线程
 */

#include <thread>
```



```

#include <vector>

void semaphore_example() {
    std::counting_semaphore<3> sem(3); // 最多3个线程同时执行

    std::vector<std::thread> threads;
    for (int i = 0; i < 10; i++) {
        threads.emplace_back([&sem, i]() {
            sem.acquire(); // 获取信号量（最多3个线程能通过）
            std::cout << "Thread " << i << " working" << std::endl;
            std::this_thread::sleep_for(std::chrono::seconds(1));
            std::cout << "Thread " << i << " done" << std::endl;
            sem.release(); // 释放信号量
        });
    }
    for (auto& t : threads) t.join();
}

```

2.10 std::barrier / std::latch (C++20)

```

#include <barrier>
#include <latch>

/*
 * std::latch:
 *  一次性倒数计数器
 *  所有线程到达后继续
 *
 * std::barrier:
 *  可重用的同步点
 *  所有线程到达后执行回调，然后重置
 */

// latch: 等待所有初始化任务完成
void latch_example() {
    std::latch work_done(3); // 等待3个任务

    for (int i = 0; i < 3; i++) {
        std::thread([&work_done, i]() {
            std::cout << "Task " << i << " initializing..." << std::endl;
            std::this_thread::sleep_for(std::chrono::seconds(1));
            std::cout << "Task " << i << " done!" << std::endl;
            work_done.count_down(); // 减1
        }).detach();
    }

    work_done.wait(); // 阻塞直到计数为0
    std::cout << "All tasks completed!" << std::endl;
}

// barrier: 多阶段同步
void barrier_example() {

```

```
const int num_threads = 3;

auto on_complete = []() noexcept {
    std::cout << "=== Phase completed! ===" << std::endl;
};

std::barrier sync_point(num_threads, on_complete);

auto worker = [&sync_point](int id) {
    for (int phase = 0; phase < 3; phase++) {
        std::cout << "Thread " << id << " phase " << phase << std::endl;
        std::this_thread::sleep_for(std::chrono::milliseconds(100 * id));
        // 等待所有线程到达
        sync_point.arrive_and_wait();
    }
};

std::vector<std::thread> threads;
for (int i = 0; i < num_threads; i++) {
    threads.emplace_back(worker, i);
}
for (auto& t : threads) t.join();
}
```

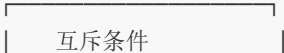
2.11 同步机制对比

机制	用途	典型场景
std::mutex	基本互斥	保护共享数据
std::recursive_mutex	可递归互斥	递归函数中的共享数据
std::timed_mutex	超时互斥	需要超时等待的场景
std::shared_mutex	读写锁	读多写少的缓存
lock_guard	RAII基本锁	简单的临界区保护
unique_lock	RAII灵活锁	条件变量、延迟锁定
scoped_lock	多锁安全获取	同时获取多把锁
condition_variable	等待/通知	生产者消费者、事件通知
semaphore	资源计数	限流、连接池
latch	一次性同步点	等待所有初始化完成
barrier	可重用同步点	多阶段并行算法

3. 死锁 (Deadlock)

3.1 死锁的四个必要条件

死锁的必要条件 (Coffman条件, 1971):

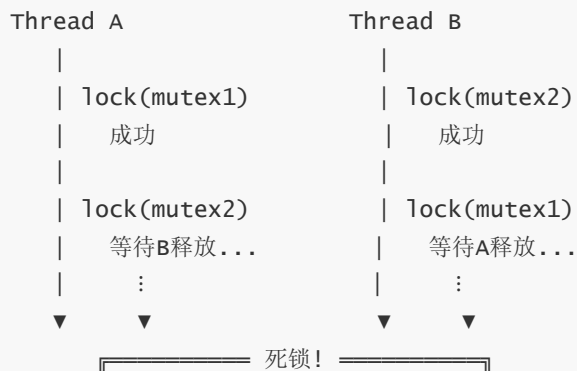


想产生死锁	(Mutual Exclusion)	资源不能共享，一次只能被一个线程使用
必须		
同时	持有并等待	
满足	(Hold and wait)	线程持有至少一个资源，又在等待其他资源
四个		
条件	不可剥夺	
	(No Preemption)	资源不能被强制抢占，只能自愿释放
	循环等待	
	(Circular wait)	存在线程-资源的循环等待链

死锁预防：破坏任一条件即可

- 破坏互斥：使用无锁数据结构（不是所有资源都能共享）
- 破坏持有并等待：一次性申请所有资源（`scoped_lock`）
- 破坏不可剥夺：`try_lock` + 回退重试
- 破坏循环等待：固定加锁顺序

死锁可视化：



3.2 预防死锁

方法1: 固定加锁顺序

```
// 始终按照相同的顺序获取锁（破坏循环等待条件）

class Account {
    std::mutex mtx;
    int id; // 账户ID，用于排序
    double balance;

public:
    void transfer(Account& to, double amount) {
        // 始终按照id顺序加锁
        Account& first = (id < to.id) ? *this : to;
        Account& second = (id < to.id) ? to : *this;

        std::lock_guard<std::mutex> lock1(first.mtx);
        std::lock_guard<std::mutex> lock2(second.mtx);

        balance -= amount;
    }
};
```

```

        to.balance += amount;
    }
};

```

方法2: try_lock 回退

```

// 尝试获取所有锁，失败则释放已持有的并重试
void transfer_try_lock(Account& to, double amount) {
    while (true) {
        if (mtx.try_lock()) {
            if (to.mtx.try_lock()) {
                // 成功获取两把锁
                balance -= amount;
                to.balance += amount;
                to.mtx.unlock();
                mtx.unlock();
                return;
            }
            mtx.unlock(); // 回退
        }
        // 随机退避，避免活锁
        std::this_thread::sleep_for(std::chrono::milliseconds(rand() % 10));
    }
}

```

方法3: scoped_lock (C++17)

```

// 最推荐的方法：使用 scoped_lock
void transfer_scoped(Account& to, double amount) {
    std::scoped_lock lock(mtx, to.mtx); // 原子地同时获取，避免死锁
    balance -= amount;
    to.balance += amount;
}

```

3.3 死锁检测

```

/*
 * 死锁检测：资源分配图
 *
 * 系统维护一个资源分配图：
 *   - 节点：线程 + 资源
 *   - 边：持有边(资源→线程) + 请求边(线程→资源)
 *   - 死锁：图中存在环
 */

// 简单的死锁检测示例
#include <atomic>

class DeadlockDetector {
    // 锁层次结构：为每个锁分配层次编号
    // 只能从低层次向高层次加锁

```

```

static thread_local int current_lock_level;

public:
class Lock {
    std::mutex mtx;
    const int level;
public:
    Lock(int lv1) : level(lv1) {}

    void lock() {
        if (current_lock_level >= level) {
            // 违反层次顺序 → 可能死锁!
            std::cerr << "POTENTIAL DEADLOCK: locking level "
                << level << " while holding level "
                << current_lock_level << std::endl;
        }
        mtx.lock();
        current_lock_level = level;
    }

    void unlock() {
        current_lock_level = level - 1;
        mtx.unlock();
    }
};

};

thread_local int DeadlockDetector::current_lock_level = 0;

```

3.4 死锁完整示例与修复

```

#include <iostream>
#include <thread>
#include <mutex>
#include <chrono>

// ===== 死锁示例 =====
void deadlock_example() {
    std::mutex m1, m2;

    std::thread t1([&]() {
        std::lock_guard<std::mutex> lk1(m1);
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
        std::lock_guard<std::mutex> lk2(m2); // 需要m2, 但m2被t2持有
        std::cout << "t1 got both locks" << std::endl;
    });

    std::thread t2([&]() {
        std::lock_guard<std::mutex> lk2(m2);
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
        std::lock_guard<std::mutex> lk1(m1); // 需要m1, 但m1被t1持有
        std::cout << "t2 got both locks" << std::endl;
    });
}

```

```

});

// 两个线程会永远阻塞!
t1.join(); // 永远不会返回
t2.join(); // 永远不会返回
std::cout << "This line is never reached!" << std::endl;
}

// ===== 修复后的代码 =====
void fixed_deadlock_example() {
    std::mutex m1, m2;

    auto safe_work = [&](const std::string& name) {
        // 方法1: std::lock 同时获取
        std::scoped_lock lock(m1, m2); // C++17, 不会死锁
        std::cout << name << " got both locks" << std::endl;
    };

    std::thread t1(safe_work, "t1");
    std::thread t2(safe_work, "t2");

    t1.join();
    t2.join();
    std::cout << "Both threads completed successfully!" << std::endl;
}

```

3.5 哲学家就餐问题

```

/*
 * 哲学家就餐问题 (Dining Philosophers)
 * 5个哲学家围坐在圆桌，每两个哲学家之间有一根筷子
 * 哲学家交替思考(thinking)和吃饭(eating)
 * 吃饭需要同时拿起左右两根筷子
 *
 * 死锁场景：
 *   所有哲学家同时拿起左筷子
 *   → 都在等待右筷子 → 死锁!
 */

#include <iostream>
#include <thread>
#include <mutex>
#include <vector>
#include <chrono>

class DiningPhilosophers {
    int num_philosophers;
    std::vector<std::mutex> forks;

public:
    DiningPhilosophers(int n) : num_philosophers(n), forks(n) {}

```

```

// 方法1: 偶数先左后右, 奇数先右后左 (破坏循环等待)
void dine_ordered(int id) {
    int left = id;
    int right = (id + 1) % num_philosophers;

    // 保证加锁顺序一致 (总是先小后大)
    int first = std::min(left, right);
    int second = std::max(left, right);

    for (int meal = 0; meal < 5; meal++) {
        // 思考
        std::cout << "Philosopher " << id << " is thinking" << std::endl;
        std::this_thread::sleep_for(std::chrono::milliseconds(100));

        // 拿筷子 (按顺序)
        std::lock_guard<std::mutex> lock1(forks[first]);
        std::lock_guard<std::mutex> lock2(forks[second]);

        // 吃饭
        std::cout << "Philosopher " << id << " is EATING (meal "
            << meal + 1 << ")" << std::endl;
        std::this_thread::sleep_for(std::chrono::milliseconds(200));
    }
}

// 方法2: scoped_lock (C++17, 最简洁)
void dine_scoped(int id) {
    int left = id;
    int right = (id + 1) % num_philosophers;

    for (int meal = 0; meal < 5; meal++) {
        std::cout << "Philosopher " << id << " is thinking" << std::endl;
        std::this_thread::sleep_for(std::chrono::milliseconds(100));

        std::scoped_lock lock(forks[left], forks[right]);

        std::cout << "Philosopher " << id << " is EATING" << std::endl;
        std::this_thread::sleep_for(std::chrono::milliseconds(200));
    }
}

void run() {
    std::vector<std::thread> philosophers;
    for (int i = 0; i < num_philosophers; i++) {
        philosophers.emplace_back(&DiningPhilosophers::dine_ordered,
            this, i);
    }
    for (auto& t : philosophers) t.join();
}
};

```

4. 锁优化

4.1 锁粒度 (Lock Granularity)

```
/*
 * 锁粒度控制原则：
 *   - 临界区越小越好（减少锁竞争）
 *   - 但也不要过度拆分（增加复杂性，可能引入死锁）
 *
 * 锁粒度的层次：
 */

// 粗粒度锁：一把锁保护所有数据
class CoarseGrainedCache {
    std::mutex mtx;
    std::map<int, int> data;
public:
    int get(int key) {
        std::lock_guard<std::mutex> lock(mtx); // 锁住整个map
        return data[key];
    }
    void put(int key, int value) {
        std::lock_guard<std::mutex> lock(mtx);
        data[key] = value;
    }
};

// 问题：get和put操作完全互斥，即使访问不同的key

// 细粒度锁：分段锁 (Segmented Lock)
class FineGrainedCache {
    static const int SEGMENTS = 16;
    std::mutex mtx[SEGMENTS];
    std::map<int, int> data[SEGMENTS];

    int segment_index(int key) {
        return std::hash<int>{}(key) % SEGMENTS;
    }
public:
    int get(int key) {
        int idx = segment_index(key);
        std::lock_guard<std::mutex> lock(mtx[idx]);
        return data[idx][key];
    }
    void put(int key, int value) {
        int idx = segment_index(key);
        std::lock_guard<std::mutex> lock(mtx[idx]);
        data[idx][key] = value;
    }
};

// 优点：不同分段的操作可以并发
// 这是ConcurrentHashMap的核心思想
```


4.2 双重检查锁定 (Double-Checked Locking)

```
/*
 * 双重检查锁定：用于延迟初始化的单例模式
 *
 * 问题：简单的加锁方案每次访问都要获取锁，性能差
 * 解决：先无锁检查，再锁内检查
 *
 * C++11之前：有内存序问题（volatile不保证原子性）
 * C++11之后：原子操作 + 正确的内存序可安全实现
 */

#include <atomic>
#include <mutex>

// C++11 正确实现：
class Singleton {
private:
    static std::atomic<Singleton*> instance;
    static std::mutex mutex;

    Singleton() = default;
public:
    static Singleton* getInstance() {
        // 第一次检查（无锁，快速路径）
        Singleton* tmp = instance.load(std::memory_order_acquire);
        if (tmp == nullptr) {
            std::lock_guard<std::mutex> lock(mutex);
            // 第二次检查（持锁）
            tmp = instance.load(std::memory_order_relaxed);
            if (tmp == nullptr) {
                tmp = new Singleton();
                instance.store(tmp, std::memory_order_release);
            }
        }
        return tmp;
    }
};

// 更简单的C++11单例（利用static局部变量的线程安全特性）
class SimpleSingleton {
public:
    static SimpleSingleton& getInstance() {
        static SimpleSingleton instance; // C++11保证线程安全！
        return instance;
    }
private:
    SimpleSingleton() = default;
};

// 注意：为什么需要volatile/原子操作？
// 编译器/CPU可能对new操作重排序：
```

```
// tmp = operator new(sizeof(Singleton)); // 分配内存
// tmp->Singleton::Singleton();           // 调用构造函数
// instance.store(tmp);                   // 赋值
//
// 如果重排序后赋值先于构造：
// 线程A：分配内存 → store → (还没构造!)
// 线程B：load → 非null → 使用未构造的对象 → 未定义行为!
//
// acquire/release内存序建立happens-before关系，防止此重排序
```

4.3 无锁编程简介

```
/*
 * 无锁编程 (Lock-Free Programming):
 * 使用原子操作代替互斥锁来保护共享数据
 *
 * 优点:
 * 1. 避免锁的开销 (等待、上下文切换)
 * 2. 避免死锁
 * 3. 避免优先级反转
 * 4. 更快的性能 (在某些场景)
 *
 * 缺点:
 * 1. 编程复杂度极高
 * 2. ABA问题
 * 3. 容易出现微妙的bug
 * 4. 不是所有算法都能无锁化
 *
 * 无锁等级:
 * wait-free: 每个操作在有限步骤内完成 (最强)
 * lock-free: 系统级保证至少一个线程可以前进
 * obstruction-free: 单线程隔离时可以前进 (最弱)
 */
```

4.4 Spinlock vs Mutex

```
#include <atomic>
#include <thread>
#include <immintrin.h>

/*
 * 自旋锁 (Spinlock):
 * 线程忙等(Busy waiting)而不是睡眠
 * 适用: 临界区非常短 (<几微秒)
 * 不适用: 临界区较长 (浪费CPU)
 *
 * 互斥锁 (Mutex):
 * 竞争时线程进入睡眠 (内核态)
 * 适用: 临界区较长
 * 不适用: 临界区极短 (上下文切换开销大于等待开销)
 */
```

```

// 简单自旋锁实现
class SimpleSpinlock {
    std::atomic_flag flag = ATOMIC_FLAG_INIT;

public:
    void lock() {
        while (flag.test_and_set(std::memory_order_acquire)) {
            // CPU释放总线（减少功耗，在超线程中让出资源给同核心的另一线程）
#ifdef __x86_64__ || defined(_M_X64)
                _mm_pause(); // x86: PAUSE指令
#elif defined(__aarch64__)
                __asm__ __volatile__("yield"); // ARM: YIELD指令
#endif
        }
    }

    void unlock() {
        flag.clear(std::memory_order_release);
    }
};

// 自适应锁（先自旋再睡眠）
class AdaptiveLock {
    std::atomic_flag flag = ATOMIC_FLAG_INIT;
    std::mutex fallback_mtx;
    static const int SPIN_COUNT = 100;

public:
    void lock() {
        // 阶段1: 自旋
        for (int i = 0; i < SPIN_COUNT; i++) {
            if (!flag.test_and_set(std::memory_order_acquire)) {
                return; // 自旋成功
            }
            _mm_pause();
        }
        // 阶段2: 睡眠（回退到mutex）
        fallback_mtx.lock();
        // 获取mutex后，尝试获取自旋锁（可能已被释放）
        while (flag.test_and_set(std::memory_order_acquire)) {
            std::this_thread::yield();
        }
        fallback_mtx.unlock();
    }

    void unlock() {
        flag.clear(std::memory_order_release);
    }
};

/*
 * Spinlock vs Mutex 对比:

```

```

*
*
* |-----|-----|-----|
* |         |      Spinlock      |      Mutex      |
* |-----|-----|-----|
* | 等待方式 | 忙等（CPU空转） | 睡眠（让出CPU） |
* | 等待开销 | CPU消耗高 | 上下文切换开销大 |
* | 最佳临界区 | < 1微秒 | > 10微秒 |
* | 内核交互 | 无（纯用户态） | 需要系统调用 |
* | 实时系统 | 适用 | 不适用 |
* | 优先级反转 | 可能发生 | 内核可处理 |
* | 功耗 | 高 | 低 |
* | 适用场景 | 短临界区、低竞争 | 长临界区、高竞争 |
*
* |-----|-----|-----|
*/

```

5. 原子操作 ★★★

5.1 std::atomic 基础

```

#include <atomic>
#include <iostream>
#include <thread>
#include <vector>

/*
 * std::atomic<T>:
 *   提供原子操作的类型包装
 *   对T的要求: trivially copyable
 *   常用特化: atomic<int>, atomic<bool>, atomic<void*>, atomic<shared_ptr>
 *
 * 核心操作:
 *   load():      原子读
 *   store():     原子写
 *   exchange():  原子交换
 *   compare_exchange_weak/strong(): CAS操作
 *   fetch_add(): 原子加
 *   fetch_sub(): 原子减
 */

// ===== 基本用法 =====

std::atomic<int> counter(0);

void atomic_increment(int n) {
    for (int i = 0; i < n; i++) {
        counter.fetch_add(1, std::memory_order_relaxed);
        // 等价于 counter++;
        // 等价于 ++counter;
    }
}
```

```

void atomic_example() {
    std::vector<std::thread> threads;
    for (int i = 0; i < 10; i++) {
        threads.emplace_back(atomic_increment, 100000);
    }
    for (auto& t : threads) t.join();

    std::cout << "Final: " << counter.load() << std::endl; // 1000000
}

// ===== atomic_flag =====

std::atomic_flag ready = ATOMIC_FLAG_INIT;

void flag_example() {
    // test_and_set: 原子地设置为true并返回旧值
    if (!ready.test_and_set()) {
        std::cout << "First thread to set flag" << std::endl;
    }

    // clear: 原子地设置为false
    ready.clear();
}

```

5.2 内存模型基础

C++11内存模型：

每个线程都有自己的视角，编译器/CPU可能对操作重排序

重排序的类型：

1. 编译器重排序：编译器优化可能改变指令顺序
2. CPU重排序：
 - Store-Load重排序（最常见，x86不允许其他类型）
 - Store-Store重排序
 - Load-Load重排序
 - Load-Store重排序

为什么需要内存序？

- 告诉编译器/CPU哪些重排序是不允许的
- 建立happens-before关系
- 确保多线程间的可见性

5.3 内存序详解 ★★★

```

/*
 * 六种内存序 (memory_order):
 *
 * relaxed:          仅保证原子性，无顺序约束
 * consume:          依赖数据的同步（少用，编译器支持不足）
 * acquire:          获取操作，之后的读写不能重排到此操作之前

```

```

* release:          释放操作，之前的读写不能重排到此操作之后
* acq_rel:          acquire + release
* seq_cst:          顺序一致性（默认），最严格，全局统一顺序
*/

#include <atomic>
#include <thread>

// =====
// 1. memory_order_relaxed
// =====
// 场景：简单计数器

std::atomic<int> relaxed_counter(0);

void relaxed_example() {
    // 多个线程增加计数器，不关心顺序
    for (int i = 0; i < 1000; i++) {
        relaxed_counter.fetch_add(1, std::memory_order_relaxed);
    }
}

// =====
// 2. memory_order_acquire / memory_order_release
// =====
// 场景：生产者-消费者（标志同步）

std::atomic<bool> data_ready(false);
int shared_data = 0; // 非原子变量

// 生产者线程
void producer() {
    shared_data = 42; // 准备数据

    // release: 保证shared_data=42的写入发生在data_ready=true之前
    //           (之前的Store不会被重排到此之后)
    data_ready.store(true, std::memory_order_release);
}

// 消费者线程
void consumer() {
    // acquire: 保证读取shared_data发生在data_ready==true之后
    //           (之后的Load不会被重排到此之前)
    while (!data_ready.load(std::memory_order_acquire))
        ;

    // 此时保证能看到 shared_data == 42
    std::cout << "shared_data = " << shared_data << std::endl;
}

/*
* release-acquire 配对:
*

```

```

*   线程A (producer)                                线程B (consumer)
*   [release之前的所有Store]                        while (!ready.acquire) ;
*   shared_data = 42;                                |
*   ready.store(true,                                ———— happens-before
*       release)                                     | [acquire之后的所有Load]
*                                                    ————> std::cout << shared_data;
*
*   release 前的所有写入 —happens-before—> acquire 后的所有读取
*/

// =====
// 3. memory_order_seq_cst (默认, 最严格)
// =====
// 场景: 需要全局一致的操作顺序

std::atomic<bool> x(false), y(false);
std::atomic<int> z(0);

void write_x() {
    x.store(true, std::memory_order_seq_cst); // (1)
}

void write_y() {
    y.store(true, std::memory_order_seq_cst); // (2)
}

void read_x_then_y() {
    while (!x.load(std::memory_order_seq_cst)) // (3)
        ;
    if (y.load(std::memory_order_seq_cst))      // (4)
        z++;
}

void read_y_then_x() {
    while (!y.load(std::memory_order_seq_cst)) // (5)
        ;
    if (x.load(std::memory_order_seq_cst))      // (6)
        z++;
}

/*
* seq_cst保证:
*   存在一个全局统一的操作顺序
*   所有线程观察到的顺序一致
*
* 对于这个例子:
*   - 如果 read_x_then_y 先看到x=true, 然后y=true → z++
*   - 如果 read_y_then_x 先看到y=true, 然后x=true → z++
*   - 所以z至少为1 (可能为2)
*
* 如果用relaxed/acq_rel, z可能为0!
*/

```

```
// =====
// 4. memory_order_acq_rel
// =====
// 场景：读-改-写操作（如fetch_add, exchange, CAS）

std::atomic<int> acq_rel_counter(0);

void acq_rel_example() {
    // fetch_add: 读旧值(acquire) + 写新值(release)
    int old = acq_rel_counter.fetch_add(1, std::memory_order_acq_rel);

    // 旧值的读有acquire语义
    // 新值的写有release语义
}

// =====
// 内存序强度对比
// =====

/*
 * 从弱到强：
 *   relaxed < consume < acquire < release < acq_rel < seq_cst
 *
 *   relaxed:    只原子，不排序，速度最快
 *   acquire:    保证后续读写可见，用于读
 *   release:    保证前序读写提交，用于写
 *   acq_rel:    acquire+release, 用于RMW
 *   seq_cst:    全局顺序一致，性能最差但最安全
 *
 * 默认是seq_cst，大多数场景用acquire/release就够了
 */
```

5.4 内存屏障/栅栏 (Memory Barrier/Fence)

```
/*
 * std::atomic_thread_fence:
 *   独立的内存屏障，不绑定特定原子变量
 *
 * std::atomic_signal_fence:
 *   仅对信号处理函数可见的内存屏障（同一线程内）
 */

#include <atomic>

std::atomic<bool> flag1(false), flag2(false);
int data = 0;

void fence_example() {
    // 线程1: 使用fence代替store的release
    data = 100;
    std::atomic_thread_fence(std::memory_order_release); // 释放屏障
    flag1.store(true, std::memory_order_relaxed);
}
```



```

    // 等价于：
    // flag1.store(true, std::memory_order_release);
}

void fence_consumer() {
    // 线程2：使用fence代替load的acquire
    while (!flag1.load(std::memory_order_relaxed))
        ;
    std::atomic_thread_fence(std::memory_order_acquire); // 获取屏障
    // 此时保证能看到 data == 100

    // 等价于：
    // while (!flag1.load(std::memory_order_acquire)) ;
}

/*
 * Fence vs Store/Load内存序：
 *
 * Store(release) = 之前的Store不能越过此Store
 *                  之后的Store/Load不能越过此Store
 *                  (单向屏障，阻止向上移动)
 *
 * Fence(release) = 之前的Store不能越过fence
 *                  之后的Store不能越过fence之前的Store
 *                  (双向屏障)
 *
 * release fence + relaxed store ≈ store(release)
 * acquire fence + relaxed load  ≈ load(acquire)
 */

```

5.5 CAS (Compare And Swap) ★★★

```

/*
 * CAS (Compare-And-Swap):
 *   如果当前值==期望值，则设置为新值
 *   否则更新期望值为当前值
 *
 * 这是lock-free编程的基石！
 *
 * C++ API:
 *   bool compare_exchange_weak(T& expected, T desired,
 *                               memory_order succ, memory_order fail);
 *   bool compare_exchange_strong(T& expected, T desired,
 *                                 memory_order succ, memory_order fail);
 *
 * weak vs strong:
 *   - weak: 可能虚假失败 (spurious failure)，即值匹配但返回false
 *           → 需要配合循环使用，在某些平台上性能更好 (如ARM的LL/SC)
 *   - strong: 保证只在值不匹配时失败
 *           → 更直观，但可能稍慢
 */

```

```

* 通用模式：
*   do {
*       expected = atomic.load();
*       desired = f(expected); // 计算新值
*   } while (!atomic.compare_exchange_weak(expected, desired));
*/

// ===== x86 cmpxchg 指令 =====
/*
* x86硬件实现：
*
* LOCK CMPXCHG [mem], reg
*
* 伪代码：
*   if (RAX == [mem]) {
*       ZF = 1;
*       [mem] = reg;    // 交换成功
*   } else {
*       ZF = 0;
*       RAX = [mem];    // 加载当前值到RAX
*   }
*
* LOCK前缀保证原子性（锁总线或缓存行）
*/

// ===== CAS示例：无锁计数器 =====
std::atomic<int> cas_counter(0);

void cas_increment() {
    int expected = cas_counter.load(std::memory_order_relaxed);
    while (!cas_counter.compare_exchange_weak(
        expected,
        expected + 1,
        std::memory_order_release,
        std::memory_order_relaxed)) {
        // expected 已被更新为当前值
        // 循环重试
    }
}

// ===== CAS示例：无锁栈 =====
template<typename T>
class LockFreeStack {
    struct Node {
        T data;
        Node* next;
        Node(const T& d) : data(d), next(nullptr) {}
    };

    std::atomic<Node*> head{nullptr};

public:
    void push(const T& data) {

```

```

Node* new_node = new Node(data);
// 设置为当前head
new_node->next = head.load(std::memory_order_relaxed);

// CAS: 如果head仍然是new_node->next, 则更新为new_node
while (!head.compare_exchange_weak(
    new_node->next,    // 期望head指向new_node->next
    new_node,         // 新head = new_node
    std::memory_order_release,
    std::memory_order_relaxed)) {
    // CAS失败, new_node->next已被更新为最新的head
    // 重试
}

bool pop(T& result) {
    Node* old_head = head.load(std::memory_order_relaxed);
    while (old_head != nullptr) {
        if (head.compare_exchange_weak(
            old_head,
            old_head->next,
            std::memory_order_acquire,
            std::memory_order_relaxed)) {
            // 成功弹出
            result = old_head->data;
            delete old_head; // 注意: 可能有ABA问题!
            return true;
        }
        // old_head已更新为重试
    }
    return false; // 栈空
}
};

```

5.6 ABA问题 ★★★

```

/*
 * ABA问题:
 *
 * 场景: 无锁栈的pop
 *
 * 初始栈: head → A → B → C
 *
 * 线程1 (正在pop):
 *   old_head = A
 *   next = B
 *   —— 中断 (上下文切换) ——
 *
 * 线程2:
 *   pop A, pop B (释放A和B)
 *   push D
 *   push A' (新分配的节点恰好复用了A的内存地址!)

```

```

*
* 现在栈: head → A' → D
*
* 线程1恢复:
*   CAS(head, A, B) → 成功! (因为head确实==A)
*   但A->next现在指向D (已经被线程2修改!)
*   结果: 栈被破坏!
*
*
* ABA问题可视化:
* 初始: head → A → B → C
* 目标: head → B → C (线程1想弹出A)
*
* 线程2处理后: head → A'(reused) → D
* 线程1 CAS成功: head → B (但B已经被释放了!)
*               实际上栈中丢失了D
*
*
* 解决方案1: 带标签的指针 (Tagged Pointer)
*/

// 带标签的指针 (x86_64上利用指针高位)
template<typename T>
struct TaggedPointer {
    T* ptr;
    uintptr_t tag; // 版本号/标签

    bool operator==(const TaggedPointer& other) const {
        return ptr == other.ptr && tag == other.tag;
    }
};

template<typename T>
class LockFreeStackABA {
    struct Node {
        T data;
        Node* next;
        Node(const T& d) : data(d), next(nullptr) {}
    };

    // 利用cmpxchg16b (128位CAS) 来原子地操作 指针+计数器
    struct alignas(16) Head {
        Node* ptr;
        uintptr_t counter;
    };

    std::atomic<Head> head; // 需要平台支持128位CAS

    // 使用double-width CAS (x86: cmpxchg16b, ARM: ldp/stp)
    // 每次修改指针时递增counter, 即使指针地址被复用, counter不同
    // 因此CAS能够检测到ABA

public:

```

```

LockFreeStackABA() {
    head.store({nullptr, 0});
}

void push(const T& data) {
    Node* new_node = new Node(data);
    Head old_head = head.load(std::memory_order_relaxed);
    Head new_head;

    do {
        new_node->next = old_head.ptr;
        new_head.ptr = new_node;
        new_head.counter = old_head.counter + 1; // 递增计数器
    } while (!head.compare_exchange_weak(
        old_head, new_head,
        std::memory_order_release,
        std::memory_order_relaxed));
}

bool pop(T& result) {
    Head old_head = head.load(std::memory_order_relaxed);
    Head new_head;

    while (old_head.ptr != nullptr) {
        new_head.ptr = old_head.ptr->next;
        new_head.counter = old_head.counter + 1;

        if (head.compare_exchange_weak(
            old_head, new_head,
            std::memory_order_acquire,
            std::memory_order_relaxed)) {
            result = old_head.ptr->data;
            delete old_head.ptr;
            return true;
        }
    }
    return false;
}

};

/*
* ABA问题的其他解决方案：
*
* 2. Hazard Pointer:
*     维护一个"危险指针"列表，标识正在使用中的指针
*     回收内存时，不立即释放，而是等到没有hazard pointer指向它
*
* 3. RCU (Read-Copy-Update):
*     读操作不加锁
*     写操作复制一份修改，然后原子更新指针
*     旧版本等到所有读者完成后释放
*
* 4. 内存回收方案：

```

```
* 使用引用计数 (std::shared_ptr + atomic)
*  → C++20: std::atomic<std::shared_ptr<T>>
*  → 或使用 std::atomic_load/store 系列函数
*/
```

6. Future与Promise

6.1 std::future / std::promise

```
#include <iostream>
#include <future>
#include <thread>
#include <chrono>

/*
 * Promise-Future 模型：
 *
 * Promise（承诺） —设置值—> Future（未来）
 *   生产者端                消费者端
 *
 * promise.set_value(x)      future.get() → x
 *
 * 工作原理：
 *   promise和future通过一个共享状态(shared state)连接
 *   shared state是引用计数的，在堆上分配
 *
 *
 *
 *
 *
 *
 *
 *
 *
 *
 *
 *
 */

// ===== 基本用法 =====

void promise_future_basic() {
    // 创建promise
    std::promise<int> promise;

    // 从promise获取future
    std::future<int> future = promise.get_future();

    // 生产者线程
    std::thread producer([&promise]() {
        std::this_thread::sleep_for(std::chrono::seconds(1));
        std::cout << "Setting value...\n";
        promise.set_value(42);          // 设置值，唤醒消费者
        // promise.set_value(100);     // 错误！只能设置一次
    });
```

```

// 消费者（主线程）
std::cout << "Waiting for value...\n";
int result = future.get(); // 阻塞直到promise设置值
std::cout << "Got: " << result << "\n";

producer.join();
// future.get()只能调用一次!
}

// ===== 异常传递 =====

void promise_exception() {
    std::promise<int> promise;
    std::future<int> future = promise.get_future();

    std::thread t([&promise]() {
        try {
            // 模拟错误
            throw std::runtime_error("Something went wrong");
        } catch (...) {
            promise.set_exception(std::current_exception());
            // 异常会被传递到 future.get()
        }
    });

    try {
        int result = future.get();
    } catch (const std::exception& e) {
        std::cout << "Caught: " << e.what() << "\n";
    }

    t.join();
}

```

6.2 std::packaged_task

```

#include <future>
#include <functional>
#include <thread>
#include <queue>

/*
 * std::packaged_task<F>:
 *   包装一个可调用对象，使其结果可以通过future获取
 *
 *   等价于：Promise + Function 的组合
 *
 *   packaged_task<R(Args...)>:
 *   - 是一个可调用对象
 *   - 调用后结果通过关联的future获取
 *   - 自动处理返回值/异常的传递
 */

```

```

*/

// ===== 基本用法 =====

int compute(int x, int y) {
    std::this_thread::sleep_for(std::chrono::seconds(1));
    return x + y;
}

void packaged_task_example() {
    // 包装 compute 函数
    std::packaged_task<int(int, int)> task(compute);

    // 获取关联的future
    std::future<int> future = task.get_future();

    // 在另一个线程中执行
    std::thread t(std::move(task), 10, 20); // packaged_task只能移动

    std::cout << "Result: " << future.get() << "\n"; // 30

    t.join();
}

// ===== 任务队列模式 =====

class TaskQueue {
    std::queue<std::packaged_task<void()>> tasks;
    std::mutex mtx;

public:
    // 提交任务并返回future
    template<typename F, typename... Args>
    auto submit(F&& f, Args&&... args)
        -> std::future<decltype(f(args...))> {

        using return_type = decltype(f(args...));

        // 包装任务
        auto task = std::make_shared<std::packaged_task<return_type()>>(
            std::bind(std::forward<F>(f), std::forward<Args>(args)...)
        );

        std::future<return_type> future = task->get_future();

        {
            std::lock_guard<std::mutex> lock(mtx);
            // 将packaged_task包装为 void() 然后放入队列
            tasks.push([task]() { (*task)(); });
        }

        return future;
    }
}

```



```

// 执行一个任务
void run_one() {
    std::packaged_task<void()> task;
    {
        std::lock_guard<std::mutex> lock(mtx);
        if (tasks.empty()) return;
        task = std::move(tasks.front());
        tasks.pop();
    }
    task();
}
};

```

6.3 std::async

```

#include <future>
#include <iostream>

/*
 * std::async:
 * 最方便的方式来异步执行任务
 *
 * 相当于: 创建线程 + packaged_task 的快捷方式
 *
 * launch policy:
 *     std::launch::async:    在新线程中异步执行 (必须)
 *     std::launch::deferred: 延迟执行, get/wait时才执行 (在同一线程)
 *     默认 (async|deferred): 由实现决定 (通常=async)
 */

// ===== async 执行策略对比 =====

int heavy_compute(int x) {
    std::cout << "Computing in thread: "
                << std::this_thread::get_id() << std::endl;
    std::this_thread::sleep_for(std::chrono::seconds(2));
    return x * x;
}

void async_example() {
    // 1. launch::async - 新线程执行
    std::future<int> f1 = std::async(std::launch::async, heavy_compute, 10);
    std::cout << "Main can do other work...\n";
    std::cout << "f1 result: " << f1.get() << "\n"; // 100

    // 2. launch::deferred - 延迟执行
    std::future<int> f2 = std::async(std::launch::deferred, heavy_compute, 20);
    std::cout << "Not executed yet...\n";
    // heavy_compute 还没被调用!
    std::cout << "f2 result: " << f2.get() << "\n"; // 现在才执行
}

```

```

// 3. 默认策略
std::future<int> f3 = std::async(heavy_compute, 30);
std::cout << "f3 result: " << f3.get() << "\n";
}

// ===== 并行计算示例 =====

#include <vector>
#include <numeric>

int parallel_sum(const std::vector<int>& data) {
    size_t n = data.size();
    size_t num_threads = std::thread::hardware_concurrency();
    size_t chunk_size = (n + num_threads - 1) / num_threads;

    std::vector<std::future<int>> futures;

    for (size_t i = 0; i < num_threads; i++) {
        size_t start = i * chunk_size;
        size_t end = std::min(start + chunk_size, n);

        if (start >= n) break;

        futures.push_back(
            std::async(std::launch::async,
                [&data, start, end]() {
                    int sum = 0;
                    for (size_t j = start; j < end; j++) {
                        sum += data[j];
                    }
                    return sum;
                })
        );
    }

    int total = 0;
    for (auto& f : futures) {
        total += f.get();
    }
    return total;
}

// ===== std::async 注意事项 =====

/*
* 陷阱1: async返回的future在析构时会阻塞!
* 如果future是async返回的临时对象, 析构函数会等待任务完成
*
* // 错误: 实际上是同步执行!
* std::async(std::launch::async, some_long_task);
* // 这里future临时对象析构, 阻塞等待任务完成
*
* // 正确:

```

```

*   auto f = std::async(std::launch::async, some_long_task);
*   // do other work...
*   f.get();
*
* 陷阱2：默认启动策略不确定
*   始终明确指定 launch::async 或 launch::deferred
*
* 陷阱3：线程数爆炸
*   async没有线程池，每次async可能创建新线程
*   大量async会导致线程过多 → 使用线程池
*/

```

6.4 std::shared_future

```

#include <future>
#include <thread>

/*
*  std::shared_future:
*   可以被多个线程等待的future
*   future只能get一次，shared_future可以多次get
*   future可以move转换为shared_future
*
* 对比：
*   future:      移动语义，独占，get一次
*   shared_future: 拷贝语义，共享，get多次
*/

void shared_future_example() {
    std::promise<int> promise;
    std::shared_future<int> shared_fut = promise.get_future().share();
    // 或者: std::shared_future<int> shared_fut(promise.get_future());

    auto worker = [shared_fut](int id) {
        int value = shared_fut.get(); // 所有线程都等待同一个结果
        std::cout << "Worker " << id << " got: " << value << "\n";
    };

    std::thread t1(worker, 1);
    std::thread t2(worker, 2);
    std::thread t3(worker, 3);

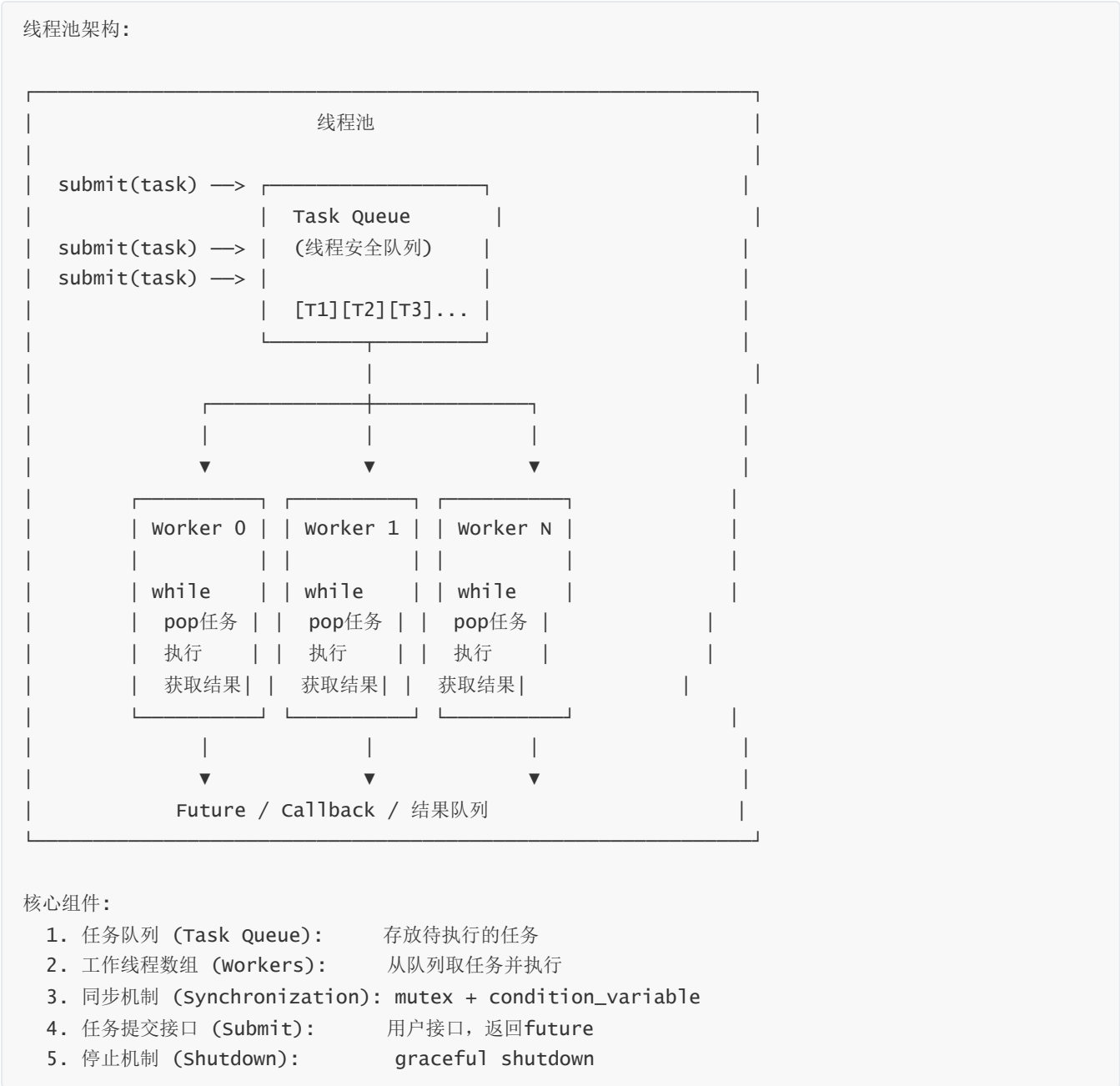
    std::this_thread::sleep_for(std::chrono::seconds(1));
    promise.set_value(42); // 所有3个线程都被唤醒

    t1.join();
    t2.join();
    t3.join();
}

```

7. 线程池 ★★★

7.1 线程池架构



7.2 完整线程池实现

```
/*
 * threadpool.h / threadpool.cpp
 * 一个功能完整的生产级线程池实现
 *
 * 特性：
 *   - 固定大小线程池
 *   - 支持任意返回类型的任务
 *   - 通过future获取结果
 *   - 优雅关闭
 *   - 异常安全
 */
```

```

*/

#include <iostream>
#include <vector>
#include <queue>
#include <thread>
#include <mutex>
#include <condition_variable>
#include <future>
#include <functional>
#include <type_traits>
#include <atomic>
#include <stdexcept>

class ThreadPool {
public:
    // 构造函数：创建指定数量的工作线程
    explicit ThreadPool(size_t num_threads)
        : stop_(false), active_tasks_(0)
    {
        for (size_t i = 0; i < num_threads; i++) {
            workers_.emplace_back([this, i] {
                // 初始化后通知（可用于调试）
                worker_loop(i);
            });
        }
    }

    // 析构函数：等待所有任务完成
    ~ThreadPool() {
        shutdown();
    }

    // 提交任务（支持各种可调用对象和参数）
    template<typename F, typename... Args>
    auto submit(F&& f, Args&&... args)
        -> std::future<typename std::invoke_result_t<F, Args...>>
    {
        using return_type = typename std::invoke_result_t<F, Args...>;

        // 创建 packaged_task，将结果与future关联
        // 使用 shared_ptr 是因为 packaged_task 不可拷贝
        auto task = std::make_shared<std::packaged_task<return_type()>>(
            std::bind(std::forward<F>(f), std::forward<Args>(args)...)
        );

        std::future<return_type> result = task->get_future();

        {
            std::unique_lock<std::mutex> lock(mutex_);

            // 不允许在已停止的线程池中提交新任务
            if (stop_) {

```

```

        throw std::runtime_error("Cannot submit to stopped ThreadPool");
    }

    // 将任务包装为 void() 放入队列
    tasks_.emplace([task]() {
        (*task)();
    });

    active_tasks_++;
}

// 唤醒一个工作线程
condition_.notify_one();

return result;
}

// 获取工作线程数量
size_t worker_count() const {
    return workers_.size();
}

// 获取待处理任务数量
size_t pending_tasks() const {
    std::unique_lock<std::mutex> lock(mutex_);
    return tasks_.size();
}

// 获取活跃任务数量（包括正在执行的和在队列中的）
size_t active_task_count() const {
    return active_tasks_.load(std::memory_order_relaxed);
}

// 等待所有任务完成（不清空队列，不停止线程池）
void wait_all() {
    while (active_tasks_.load(std::memory_order_acquire) > 0) {
        std::this_thread::yield();
    }
}

// 优雅关闭：完成队列中所有任务后退出
void shutdown() {
    {
        std::unique_lock<std::mutex> lock(mutex_);
        if (stop_) return; // 防止重复关闭
        stop_ = true;
    }

    // 唤醒所有等待的工作线程
    condition_.notify_all();

    // 等待所有线程退出
    for (std::thread& worker : workers_) {

```

```

        if (worker.joinable()) {
            worker.join();
        }
    }

    // 清空剩余任务（如果有的话）
    std::queue<std::function<void()>> empty;
    {
        std::unique_lock<std::mutex> lock(mutex_);
        std::swap(tasks_, empty);
    }
}

// 立即关闭：清空队列，停止所有线程
void shutdown_now() {
    {
        std::unique_lock<std::mutex> lock(mutex_);
        stop_ = true;
        // 清空任务队列
        std::queue<std::function<void()>> empty;
        std::swap(tasks_, empty);
        active_tasks_ = 0;
    }

    condition_.notify_all();

    for (std::thread& worker : workers_) {
        if (worker.joinable()) {
            worker.join();
        }
    }
}

// 禁止拷贝
ThreadPool(const ThreadPool&) = delete;
ThreadPool& operator=(const ThreadPool&) = delete;
ThreadPool(ThreadPool&&) = delete;
ThreadPool& operator=(ThreadPool&&) = delete;

private:
    // 工作线程的主循环
    void worker_loop(int worker_id) {
        while (true) {
            std::function<void()> task;

            {
                std::unique_lock<std::mutex> lock(mutex_);

                // 等待任务或停止信号
                condition_.wait(lock, [this] {
                    return stop_ || !tasks_.empty();
                });
            }
        }
    }
}

```

```

        // 停止且队列为空 → 退出
        if (stop_ && tasks_.empty()) {
            return;
        }

        // 取出任务
        task = std::move(tasks_.front());
        tasks_.pop();
    }

    // 执行任务（锁外执行，不阻塞其他线程取任务）
    task();
    active_tasks_--;
}

}

// 成员变量
std::vector<std::thread> workers_;           // 工作线程
std::queue<std::function<void()>> tasks_;    // 任务队列
mutable std::mutex mutex_;                  // 互斥锁
std::condition_variable condition_;         // 条件变量
bool stop_;                                 // 停止标志
std::atomic<size_t> active_tasks_;          // 活跃任务计数
};

// ===== 使用示例 =====

void threadpool_example() {
    // 创建线程池（4个线程）
    ThreadPool pool(4);

    // 提交各种任务
    std::vector<std::future<int>> results;

    for (int i = 0; i < 20; i++) {
        results.push_back(
            pool.submit([i]() -> int {
                // 模拟计算
                std::this_thread::sleep_for(
                    std::chrono::milliseconds(100));
                return i * i;
            })
        );
    }

    // 获取结果
    for (int i = 0; i < (int)results.size(); i++) {
        std::cout << "Task " << i << " result: "
                    << results[i].get() << std::endl;
    }

    // 提交带参数的任务
    auto f = pool.submit([](int a, int b, int c) {

```



```

        return a + b + c;
    }, 1, 2, 3);
std::cout << "Sum: " << f.get() << std::endl; // 6

// 提交 void 任务
pool.submit([]() {
    std::cout << "Void task executed" << std::endl;
});

// 析构时自动调用 shutdown()
}

```

7.3 带优先级的线程池

```

/*
 * PriorityThreadPool: 支持任务优先级的线程池
 *
 * 注意: 这个实现使用两个队列, 优先处理高优先级任务
 */

#include <queue>
#include <functional>

class PriorityThreadPool {
public:
    enum class Priority { HIGH = 0, LOW = 1 };

    PriorityThreadPool(size_t num_threads) : stop_(false) {
        for (size_t i = 0; i < num_threads; i++) {
            workers_.emplace_back([this] { worker_loop(); });
        }
    }

    ~PriorityThreadPool() {
        shutdown();
    }

    template<typename F, typename... Args>
    auto submit(Priority priority, F&& f, Args&&... args)
        -> std::future<typename std::invoke_result_t<F, Args...>>
    {
        using return_type = typename std::invoke_result_t<F, Args...>;

        auto task = std::make_shared<std::packaged_task<return_type()>>(
            std::bind(std::forward<F>(f), std::forward<Args>(args)...)
        );

        std::future<return_type> result = task->get_future();

        {
            std::unique_lock<std::mutex> lock(mutex_);

```

```

        if (stop_) {
            throw std::runtime_error("Pool is stopped");
        }

        if (priority == Priority::HIGH) {
            high_priority_tasks_.emplace([task] { (*task)(); });
        } else {
            low_priority_tasks_.emplace([task] { (*task)(); });
        }
    }

    condition_.notify_one();
    return result;
}

void shutdown() {
    {
        std::unique_lock<std::mutex> lock(mutex_);
        stop_ = true;
    }
    condition_.notify_all();
    for (auto& w : workers_) {
        if (w.joinable()) w.join();
    }
}

private:
void worker_loop() {
    while (true) {
        std::function<void()> task;

        {
            std::unique_lock<std::mutex> lock(mutex_);

            condition_.wait(lock, [this] {
                return stop_ ||
                    !high_priority_tasks_.empty() ||
                    !low_priority_tasks_.empty();
            });

            if (stop_ && high_priority_tasks_.empty()
                && low_priority_tasks_.empty()) {
                return;
            }

            // 优先取高优先级任务
            if (!high_priority_tasks_.empty()) {
                task = std::move(high_priority_tasks_.front());
                high_priority_tasks_.pop();
            } else if (!low_priority_tasks_.empty()) {
                task = std::move(low_priority_tasks_.front());
                low_priority_tasks_.pop();
            }
        }
    }
}

```

```

    }

    task();
}
}

std::vector<std::thread> workers_;
std::queue<std::function<void()>> high_priority_tasks_;
std::queue<std::function<void()>> low_priority_tasks_;
std::mutex mutex_;
std::condition_variable condition_;
bool stop_;
};

```

7.4 动态扩缩容线程池

```

/*
 * DynamicThreadPool: 支持动态调整线程数量的线程池
 *
 * 核心思想:
 *   - 维护 核心线程数 和 最大线程数
 *   - 空闲时缩减到核心线程数
 *   - 任务堆积时扩展到最大线程数
 */

class DynamicThreadPool {
public:
    DynamicThreadPool(size_t core_threads, size_t max_threads,
                      std::chrono::milliseconds idle_timeout
                          = std::chrono::seconds(60))
        : core_threads_(core_threads)
        , max_threads_(max_threads)
        , idle_timeout_(idle_timeout)
        , current_threads_(0)
        , stop_(false)
    {
        // 创建核心线程
        for (size_t i = 0; i < core_threads; i++) {
            add_worker();
        }
    }

    ~DynamicThreadPool() {
        shutdown();
    }

    template<typename F, typename... Args>
    auto submit(F&& f, Args&&... args)
        -> std::future<typename std::invoke_result_t<F, Args...>>
    {
        using return_type = typename std::invoke_result_t<F, Args...>;

```

```

    auto task = std::make_shared<std::packaged_task<return_type()>>(
        std::bind(std::forward<F>(f), std::forward<Args>(args)...)
    );

    std::future<return_type> result = task->get_future();

    {
        std::unique_lock<std::mutex> lock(mutex_);

        if (stop_) {
            throw std::runtime_error("Pool is stopped");
        }

        tasks_.emplace([task] { (*task)(); });

        // 动态扩容：如果队列堆积且线程数未达到上限
        if (tasks_.size() > current_threads_ &&
            current_threads_ < max_threads_) {
            add_worker();
        }
    }

    condition_.notify_one();
    return result;
}

size_t thread_count() const {
    std::unique_lock<std::mutex> lock(mutex_);
    return current_threads_;
}

void shutdown() {
    {
        std::unique_lock<std::mutex> lock(mutex_);
        stop_ = true;
    }
    condition_.notify_all();

    for (auto& w : workers_) {
        if (w.joinable()) w.join();
    }
}

private:
void add_worker() {
    workers_.emplace_back([this] {
        auto thread_start = std::chrono::steady_clock::now();
        current_threads_++;

        while (true) {
            std::function<void()> task;

            {

```

```

        std::unique_lock<std::mutex> lock(mutex_);

        // 空闲超时（非核心线程可以退出）
        auto timeout = (current_threads_ > core_threads_)
            ? idle_timeout_
            : std::chrono::milliseconds::max();

        bool has_task = condition_.wait_for(
            lock, timeout,
            [this] { return stop_ || !tasks_.empty(); }
        );

        if (stop_ && tasks_.empty()) {
            current_threads_--;
            return;
        }

        if (!has_task) {
            // 超时，非核心线程退出
            if (current_threads_ > core_threads_) {
                current_threads_--;
                return;
            }
            continue; // 核心线程继续等待
        }

        task = std::move(tasks_.front());
        tasks_.pop();
    }

    task(); // 执行任务
}

});
}

size_t core_threads_;
size_t max_threads_;
std::chrono::milliseconds idle_timeout_;
std::atomic<size_t> current_threads_;
std::atomic<bool> stop_;
std::vector<std::thread> workers_;
std::queue<std::function<void()>> tasks_;
mutable std::mutex mutex_;
std::condition_variable condition_;
};

```

8. 生产者消费者模型

8.1 有界阻塞队列 (Bounded Buffer)

```
/*
 * 经典的生产者-消费者模型
 * 使用 互斥锁 + 两个条件变量
 *
 * not_full: 队列不满时通知生产者
 * not_empty: 队列不空时通知消费者
 *
 * BoundedBuffer (有界缓冲区):
 * - 固定容量的环形缓冲区
 * - 队满时生产者阻塞
 * - 队空时消费者阻塞
 */

#include <queue>
#include <mutex>
#include <condition_variable>
#include <thread>
#include <iostream>
#include <vector>

template<typename T>
class BoundedBuffer {
private:
    std::queue<T> queue_;
    size_t capacity_;
    mutable std::mutex mutex_;
    std::condition_variable not_full_; // 生产者等待
    std::condition_variable not_empty_; // 消费者等待

public:
    explicit BoundedBuffer(size_t capacity) : capacity_(capacity) {}

    // 生产者：放入数据
    void put(const T& item) {
        std::unique_lock<std::mutex> lock(mutex_);

        // 队列满 → 等待 not_full
        not_full_.wait(lock, [this] {
            return queue_.size() < capacity_;
        });

        queue_.push(item);

        // 放入后通知消费者
        not_empty_.notify_one();
    }

    // 消费者：取出数据
```

```

T take() {
    std::unique_lock<std::mutex> lock(mutex_);

    // 队列空 → 等待 not_empty
    not_empty_.wait(lock, [this] {
        return !queue_.empty();
    });

    T item = queue_.front();
    queue_.pop();

    // 取出后通知生产者
    not_full_.notify_one();

    return item;
}

// 非阻塞尝试放入
bool try_put(const T& item) {
    std::lock_guard<std::mutex> lock(mutex_);
    if (queue_.size() >= capacity_) {
        return false;
    }
    queue_.push(item);
    not_empty_.notify_one();
    return true;
}

// 非阻塞尝试取出
bool try_take(T& item) {
    std::lock_guard<std::mutex> lock(mutex_);
    if (queue_.empty()) {
        return false;
    }
    item = queue_.front();
    queue_.pop();
    not_full_.notify_one();
    return true;
}

// 获取当前大小
size_t size() const {
    std::lock_guard<std::mutex> lock(mutex_);
    return queue_.size();
}
};

// ===== 使用示例：单生产者单消费者 =====

void spsc_example() {
    BoundedBuffer<int> buffer(5); // 容量5

    // 生产者

```

```

std::thread producer([&buffer]() {
    for (int i = 0; i < 20; i++) {
        buffer.put(i);
        std::cout << "Produced: " << i << std::endl;
        std::this_thread::sleep_for(std::chrono::milliseconds(50));
    }
});

// 消费者
std::thread consumer([&buffer]() {
    for (int i = 0; i < 20; i++) {
        int item = buffer.take();
        std::cout << "Consumed: " << item << std::endl;
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
    }
});

producer.join();
consumer.join();
}

// ===== 使用示例：多生产者多消费者 =====

void mpmc_example() {
    BoundedBuffer<int> buffer(10);

    const int num_producers = 3;
    const int num_consumers = 2;
    const int items_per_producer = 10;

    std::vector<std::thread> producers;
    std::vector<std::thread> consumers;
    std::atomic<int> total_consumed{0};

    for (int i = 0; i < num_producers; i++) {
        producers.emplace_back([&buffer, i, items_per_producer]() {
            for (int j = 0; j < items_per_producer; j++) {
                buffer.put(i * 100 + j);
                std::cout << "Producer " << i << " -> " << (i * 100 + j) << "\n";
            }
        });
    }

    for (int i = 0; i < num_consumers; i++) {
        consumers.emplace_back([&buffer, &total_consumed]() {
            int total = num_producers * items_per_producer / num_consumers;
            for (int j = 0; j < total; j++) {
                int item = buffer.take();
                total_consumed++;
                std::cout << "Consumer -> " << item << "\n";
            }
        });
    }
}

```



```

    for (auto& t : producers) t.join();
    for (auto& t : consumers) t.join();
}

```

8.2 无锁SPSC队列 (Lock-Free SPSC Queue)

```

/*
 * 无锁单生产者单消费者队列 (Lock-Free SPSC Queue)
 *
 * 这是性能最高的队列类型
 * 利用缓存行对齐避免伪共享 (false sharing)
 * 使用原子操作代替锁
 */

#include <atomic>
#include <vector>
#include <new>

template<typename T>
class LockFreeSPSCQueue {
private:
    // 缓存行对齐 (通常64字节)
    // 避免 head 和 tail 在同一个缓存行导致伪共享
    static constexpr size_t CACHE_LINE_SIZE = 64;

    struct alignas(CACHE_LINE_SIZE) {
        std::atomic<size_t> head{0};
    };

    struct alignas(CACHE_LINE_SIZE) {
        std::atomic<size_t> tail{0};
    };

    // 无锁SPSC只需要两个独立的原子变量
    // head: 消费者使用
    // tail: 生产者使用
    // 它们被不同的缓存行隔开 → 没有伪共享
    alignas(CACHE_LINE_SIZE) std::atomic<size_t> head_{0};
    alignas(CACHE_LINE_SIZE) std::atomic<size_t> tail_{0};

    const size_t capacity_;
    T* buffer_; // 使用 placement new 构造

    // 2的幂大小便于使用位运算取模
    static constexpr size_t round_up_pow2(size_t n) {
        size_t p = 1;
        while (p < n) p <= 1;
        return p;
    }

public:

```

```

explicit LockFreeSPSCQueue(size_t capacity)
    : capacity_(round_up_pow2(capacity))
{
    // 原始内存分配
    buffer_ = static_cast<T*>(
        ::operator new(capacity_ * sizeof(T), std::align_val_t(alignof(T)))
    );
}

~LockFreeSPSCQueue() {
    // 销毁队列中剩余的元素
    size_t h = head_.load(std::memory_order_relaxed);
    size_t t = tail_.load(std::memory_order_relaxed);
    while (h != t) {
        buffer_[h & (capacity_ - 1)].~T();
        h++;
    }
    ::operator delete(buffer_, capacity_ * sizeof(T),
        std::align_val_t(alignof(T)));
}

// 禁止拷贝
LockFreeSPSCQueue(const LockFreeSPSCQueue&) = delete;
LockFreeSPSCQueue& operator=(const LockFreeSPSCQueue&) = delete;

// 生产者：尝试放入（非阻塞）
bool try_push(const T& value) {
    size_t tail = tail_.load(std::memory_order_relaxed);
    size_t head = head_.load(std::memory_order_acquire);

    // 检查是否满（用容量掩码代替取模）
    size_t mask = capacity_ - 1;
    if (tail - head >= capacity_) {
        return false; // 队列满
    }

    // 在对应位置构造元素
    new (&buffer_[tail & mask]) T(value);

    // 更新tail，释放语义确保元素构造在tail更新前完成
    tail_.store(tail + 1, std::memory_order_release);
    return true;
}

// 生产者：放入（阻塞，简单自旋）
void push(const T& value) {
    while (!try_push(value)) {
        std::this_thread::yield();
    }
}

// 消费者：尝试取出（非阻塞）
bool try_pop(T& result) {

```

```

size_t head = head_.load(std::memory_order_relaxed);
size_t tail = tail_.load(std::memory_order_acquire);

// 检查是否空
if (head == tail) {
    return false; // 队列空
}

size_t mask = capacity_ - 1;
result = std::move(buffer_[head & mask]);

// 销毁被消费的元素
buffer_[head & mask].~T();

// 更新head, 释放语义
head_.store(head + 1, std::memory_order_release);
return true;
}

// 消费者: 取出 (阻塞, 简单自旋)
T pop() {
    T result;
    while (!try_pop(result)) {
        std::this_thread::yield();
    }
    return result;
}

size_t size() const {
    size_t head = head_.load(std::memory_order_acquire);
    size_t tail = tail_.load(std::memory_order_acquire);
    return tail - head;
}

bool empty() const {
    return size() == 0;
}

size_t capacity() const {
    return capacity_;
}
};

/*
 * 伪共享 (False Sharing) 详解:
 *
 * CPU缓存以缓存行(Cache Line, 通常64字节)为单位
 * 两个独立变量在同一个缓存行中:
 *
 * ┌────────── 缓存行 (64 bytes) ─────────┐
 * │ var1 (8B) │ var2 (8B) │ unused (48B) │
 * └────────────────────────────────────────┘
 */

```

```

* Core 0 修改 var1 → 整个缓存行失效
* Core 1 读取 var2 → 缓存未命中！（实际var2没变）
*
* 这就是伪共享 – 不同的变量因为恰好在同一缓存行而产生竞争
* 解决：使用 alignas(64) 将变量分隔到不同缓存行
*
* SPSC的巧妙之处：
*   生产者只写tail，消费者只写head
*   它们是完全独立的两个变量
*   只要不在同一缓存行，就没有任何竞争！
*/

```

9. 并发数据结构

9.1 线程安全队列

```

/*
 * ThreadSafeQueue: 线程安全的有界队列
 *
 * 特性：
 *   - 支持多生产者多消费者（MPMC）
 *   - 有界（容量有限）
 *   - 阻塞和非阻塞操作
 */

#include <queue>
#include <mutex>
#include <condition_variable>

template<typename T>
class ThreadSafeQueue {
private:
    mutable std::mutex mutex_;
    std::condition_variable not_empty_;
    std::condition_variable not_full_;
    std::queue<T> queue_;
    size_t capacity_;
    bool closed_;

public:
    explicit ThreadSafeQueue(size_t capacity = SIZE_MAX)
        : capacity_(capacity), closed_(false) {}

    // 放入（阻塞直到有空间或关闭）
    bool push(T item) {
        std::unique_lock<std::mutex> lock(mutex_);
        not_full_.wait(lock, [this] {
            return queue_.size() < capacity_ || closed_;
        });

        if (closed_) return false;
    }

```

```

        queue_.push(std::move(item));
        not_empty_.notify_one();
        return true;
    }

    // 取出（阻塞直到有元素或关闭且为空）
    bool pop(T& item) {
        std::unique_lock<std::mutex> lock(mutex_);
        not_empty_.wait(lock, [this] {
            return !queue_.empty() || (queue_.empty() && closed_);
        });

        if (queue_.empty() && closed_) return false;

        item = std::move(queue_.front());
        queue_.pop();
        not_full_.notify_one();
        return true;
    }

    // 尝试放入（非阻塞）
    bool try_push(T item) {
        std::lock_guard<std::mutex> lock(mutex_);
        if (queue_.size() >= capacity_ || closed_) {
            return false;
        }
        queue_.push(std::move(item));
        not_empty_.notify_one();
        return true;
    }

    // 尝试取出（非阻塞）
    bool try_pop(T& item) {
        std::lock_guard<std::mutex> lock(mutex_);
        if (queue_.empty()) {
            return false;
        }
        item = std::move(queue_.front());
        queue_.pop();
        not_full_.notify_one();
        return true;
    }

    // 关闭队列（唤醒所有等待的线程）
    void close() {
        std::lock_guard<std::mutex> lock(mutex_);
        closed_ = true;
        not_empty_.notify_all();
        not_full_.notify_all();
    }

    bool is_closed() const {

```

```

        std::lock_guard<std::mutex> lock(mutex_);
        return closed_;
    }

    size_t size() const {
        std::lock_guard<std::mutex> lock(mutex_);
        return queue_.size();
    }

    bool empty() const {
        std::lock_guard<std::mutex> lock(mutex_);
        return queue_.empty();
    }
};

```

9.2 线程安全栈

```

/*
 * ThreadSafeStack: 线程安全的栈
 *
 * 注意: 栈的top和pop是分开操作
 *      在多线程环境中有竞态条件
 *      (top后另一个线程可能已经把元素pop了)
 *
 * 解决方案: 将top和pop合并为一个操作
 */

#include <stack>
#include <mutex>
#include <memory>
#include <exception>

struct EmptyStackException : std::exception {
    const char* what() const noexcept override {
        return "Stack is empty";
    }
};

template<typename T>
class ThreadSafeStack {
private:
    std::stack<T> stack_;
    mutable std::mutex mutex_;

public:
    ThreadSafeStack() = default;
    ThreadSafeStack(const ThreadSafeStack& other) {
        std::lock_guard<std::mutex> lock(other.mutex_);
        stack_ = other.stack_;
    }

    ThreadSafeStack& operator=(const ThreadSafeStack&) = delete;

```

```

void push(T value) {
    std::lock_guard<std::mutex> lock(mutex_);
    stack_.push(std::move(value));
}

// 合并 top + pop 的线程安全版本
// 返回值为 shared_ptr，避免空栈时的异常问题
std::shared_ptr<T> pop() {
    std::lock_guard<std::mutex> lock(mutex_);
    if (stack_.empty()) {
        throw EmptyStackException();
    }
    // 先取到值再弹出
    auto result = std::make_shared<T>(std::move(stack_.top()));
    stack_.pop();
    return result;
}

// 或者：top + pop 一起，结果存入引用参数
bool pop(T& value) {
    std::lock_guard<std::mutex> lock(mutex_);
    if (stack_.empty()) {
        return false;
    }
    value = std::move(stack_.top());
    stack_.pop();
    return true;
}

bool empty() const {
    std::lock_guard<std::mutex> lock(mutex_);
    return stack_.empty();
}

size_t size() const {
    std::lock_guard<std::mutex> lock(mutex_);
    return stack_.size();
}
};

```

10. 面试总结

10.1 面试重点

知识点	重要度	说明
std::atomic + 内存序	★★★★★	大厂面试必问，需要能讲清楚acquire/release
CAS原理和ABA问题	★★★★★	无锁编程的核心，必须掌握

知识点	重要度	说明
线程池设计	★★★★★	手写代码题，要能写出完整实现
死锁条件与预防	★★★★★	基础理论，必须掌握
条件变量的正确使用	★★★★	wait+谓词、虚假唤醒、丢失唤醒
mutex系列	★★★★	lock_guard / unique_lock / scoped_lock
读写锁 shared_mutex	★★★★	读多写少场景、实现原理
生产者消费者	★★★★	经典模型，面试高频
future/promise	★★★★	异步编程基础
Double-Checked Locking	★★★	单例模式、内存序原因
Spinlock vs Mutex	★★★	适用场景分析
lock-free vs wait-free	★★★	无锁编程级别

10.2 高频考点

1. std::atomic 有哪几种内存序？acquire/release 怎么用？

→ seq_cst(默认)/acquire/release/acq_rel/relaxed。acquire用于读，release用于写，配对使用建立happens-before关系。

2. 什么是ABA问题？怎么解决？

→ CAS中值A被改为B再改回A，CAS误判为未变化。用tagged pointer、hazard pointer、引用计数解决。

3. 死锁的四个必要条件？怎么预防？

→ 互斥、持有并等待、不可剥夺、循环等待。预防手段：scoped_lock、固定加锁顺序、try_lock回退。

4. condition_variable的wait为什么需要lock？虚假唤醒是什么？

→ 原子地释放锁+进入等待，避免丢失唤醒。虚假唤醒是POSIX允许的，需要在while循环中检查条件。

5. lock_guard vs unique_lock vs scoped_lock？

→ lock_guard最简单；unique_lock可延迟/手动/转移；scoped_lock可同时锁多个mutex且避免死锁。

6. 怎么设计一个线程池？

→ 任务队列 + 工作线程 + 条件变量通知 + future返回结果 + 优雅关闭。

10.3 常见陷阱

1. 不使用带谓词的condition_variable::wait → 虚假唤醒导致错误

2. 忘记join或detach线程 → std::terminate

3. 在持有锁时调用用户代码 → 可能导致死锁

4. 默认内存序seq_cst的性能问题 → 很多场景relaxed/acq_rel就够了

5. Double-Checked Locking不用atomic → 未定义行为

6. lock_guard生命周期太短/太长 → 临界区范围不对

7. 非atomic变量在多线程中读/写 → 数据竞争

8. detached线程访问栈上变量 → 悬空引用

9. **shared_ptr不是线程安全的** → 需要atomic_load/store (C++20有atomic<shared_ptr>)

10. **async返回的临时future析构阻塞** → 保存future或使用线程池

10.4 建议背诵内容

1. 六种内存序的名称和用途
2. CAS的C++ API和典型使用模式
3. ABA问题的描述和解决方案
4. 死锁的四个必要条件和三种预防方法
5. 完整线程池的代码框架 (至少能写出submit方法和worker循环)
6. condition_variable带谓词wait的用法
7. unique_lock / lock_guard / scoped_lock 的区别
8. 生产者消费者有界队列的代码
9. Double-Checked Locking的正确atomic实现
10. Happens-Before关系图